

Python : Orienté objets

Tables des matières

- | | |
|----------------------|-----------------------|
| 1. Introduction | 7. Polymorphisme |
| 2. Structures | 8. Classes abstraites |
| 3. Classes et objets | 9. Membres statiques |
| 4. Encapsulation | 10. Interfaces |
| 5. Data model | 11. Constructeurs |
| 6. Héritage | |

1. Introduction

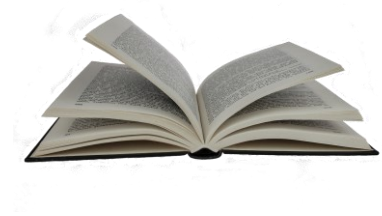
- Définition de la programmation orientée objet
- Comparaison avec la programmation procédurale
- Principes fondamentaux
- Historique et évolution
- Importance de la POO

Définition de la programmation orientée objet

La POO offre une approche de programmation qui repose sur la modélisation des entités du monde réel sous forme d'objets, chacun possédant des caractéristiques (**attributs**) et des actions (**méthodes**) qui lui sont propres.



Cette approche permet de structurer le code de manière logique et intuitive, en favorisant la réutilisabilité, la modularité et la maintenance aisée du code. En permettant de représenter des entités complexes sous forme d'objets autonomes interagissant entre eux.



La POO permet à un objet de prendre pratiquement n'importe quelle forme, ce qui rend la modélisation des concepts du monde réel ou fictif plus flexible et puissante.



Comparaison avec la programmation procédurale

Contrairement à la programmation procédurale, qui divise le code en séquences d'instructions, la POO organise le code autour de données et de fonctions qui agissent sur ces données, rassemblées au sein d'objets.

Cette approche permet une meilleure modélisation des concepts du monde réel, en favorisant la compréhension du code et en réduisant les risques d'erreurs.

La POO encourage la réutilisation du code grâce à la création de bibliothèques d'objets réutilisables, ce qui accélère le processus de développement et améliore la qualité du code produit.

Procédurale :

```
1 def creer_chat(nom, couleur):
2     return {"nom": nom, "couleur": couleur}
3
4 def miauler(chat):
5     return f"{chat['nom']} miaule."
6
7 # Création d'un chat et appel de la fonction miauler
8 mon_chat = creer_chat("Whiskers", "gris")
9 print(miauler(mon_chat)) # Affiche : Whiskers miaule.
```

POO :

```
1 class Chat:
2     def __init__(self, nom, couleur):
3         self.nom = nom
4         self.couleur = couleur
5
6     def miauler(self):
7         return f"{self.nom} miaule."
8
9 # Création d'un objet chat
10 mon_chat = Chat("Whiskers", "gris")
11 print(mon_chat.miauler()) # Affiche : Whiskers miaule.
```

Principes fondamentaux

Outre les concepts d'encapsulation, d'héritage et de polymorphisme qui seront vu dans les chapitre suivants, la POO repose sur le principe d'abstraction, qui consiste à isoler les aspects essentiels d'un objet tout en cachant les détails d'implémentation superflus.

Cette abstraction permet de simplifier la complexité du système en fournissant des interfaces claires et cohérentes pour interagir avec les objets.

En combinant ces principes, la POO offre un cadre flexible et extensible pour la conception de logiciels complexes, facilitant la maintenance et l'évolution du code au fil du temps.



Historiques et évolution de la POO

La POO trouve ses origines dans les travaux pionniers réalisés dans les années 1960 avec le langage de programmation Simula, qui a introduit les concepts de classes et d'objets.

Depuis lors, la POO a connu un développement rapide avec l'émergence de langages tels que Smalltalk, C++, Java et Python, qui ont étendu et popularisé les concepts de la POO. Aujourd'hui, celle-ci est largement utilisée dans le développement logiciel pour sa capacité à modéliser des systèmes complexes comme le montre le tableau suivant.

Année	Événement
1960	Simula, le premier langage de programmation à introduire les concepts de classes et d'objets, est développé.
Années 1970	Smalltalk émerge comme un langage de programmation influent popularisant les idées de la POO, y compris les classes, les objets et l'encapsulation.
Années 1980	C++, un langage de programmation hybride combinant la programmation procédurale avec des concepts de la POO comme l'héritage et le polymorphisme, est créé.

Historiques et évolution de la POO

Années 1990	Java est lancé, un langage de programmation orienté objet conçu pour être portable et sûr, contribuant à la popularisation de la POO.
Années 2000	Python gagne en popularité comme un langage de programmation polyvalent avec un support intégré pour la POO, offrant une syntaxe propre et une grande flexibilité.
Années 2010	L'importance de la POO dans l'ingénierie logicielle moderne est encore renforcée par l'essor des méthodologies de développement agile et des pratiques de développement centrées sur les objets.
Années 2020	La POO continue d'évoluer avec l'émergence de nouvelles tendances telles que la programmation orientée objet fonctionnelle et les architectures logicielles basées sur les microservices.
2024	Les principes de la POO demeurent fondamentaux dans la conception et le développement de logiciels, contribuant à la création de systèmes évolutifs, flexibles et maintenables.

Importance de la POO

Au-delà de ses avantages techniques, la POO favorise des pratiques de développement plus collaboratives et agiles, en encourageant la modularité, la réutilisabilité et la maintenabilité du code.

En permettant de représenter des entités du monde réel de manière naturelle, la POO facilite la communication entre les membres de l'équipe de développement et favorise une meilleure compréhension des exigences du projet.

Son impact s'étend également au-delà du développement logiciel, influençant des domaines tels que l'analyse des données, l'intelligence artificielle et l'ingénierie logicielle.

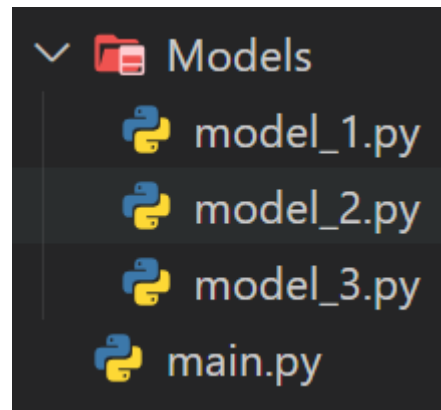
2. Structures

- Travailler avec plusieurs fichiers
- Logique d'imports
- Limite d'exécution
- Bonnes pratiques

Travailler avec plusieurs fichiers

Il est important de structurer le code sur plusieurs fichiers afin de séparer les différentes classes et fonctionnalités. Chaque fichier peut contenir des classes et des méthodes associées à des aspects spécifiques du projet.

Cette approche favorise la modularité et la réutilisabilité du code, en nous permettant de se concentrer sur des parties spécifiques de l'application.



Logique d'import

L'instruction `from Models.model_1 import model_1` est une façon de charger spécifiquement une classe ou une fonction appelée `model_1` depuis un fichier Python appelé `model_1.py` situé dans le répertoire `Models`.

Cette méthode d'importation permet d'accéder directement à la classe ou à la fonction sans avoir besoin de spécifier le nom du module lors de son utilisation. Ensuite, en utilisant `test = model_1()`, nous créons une instance de la classe `model_1`, prête à être utilisée dans notre programme.

Ce type d'importation sélective est couramment utilisé dans les projets pour simplifier l'accès à des fonctionnalités spécifiques et rendre le code plus lisible et modulaire.

```
1  from Models.model_1 import model_1
2  test = model_1()
```

Limite d'exécution

Lors de l'importation d'un fichier Python, il est souvent nécessaire de restreindre l'exécution du code contenu dans ce fichier.

Cela peut être réalisé en utilisant le nom du scope, accessible via la variable spéciale `__name__`. Lorsque le fichier est exécuté directement en tant que point d'entrée, la valeur de `__name__` est définie sur "main".

Ainsi, en vérifiant si `__name__` est égal à "main", nous pouvons conditionner l'exécution de certaines parties du code à cet état.

```
1  def fonction_principale():
2      print("Code exécuté uniquement si le fichier est le point d'entrée")
3
4  if __name__ == "__main__":
5      fonction_principale()
```

Bonnes pratiques

Pour maintenir une structure de projet cohérente et efficace, il est recommandé d'adopter certaines bonnes pratiques :

- **Utiliser des noms descriptifs** : Choisissez des noms de classe qui reflètent clairement le rôle ou la fonction de la classe dans votre système. Les noms descriptifs facilitent la compréhension du code par d'autres développeurs.
- **Éviter les abréviations cryptiques** : Évitez les abréviations peu claires ou cryptiques dans les noms de classe. Optez plutôt pour des noms complets et explicites qui rendent le code plus compréhensible.
- **Respecter la convention de nommage PascalCase** : Suivez la convention de nommage **PascalCase** pour les noms de classe. Cela contribue à maintenir la cohérence dans le code Python et facilite la collaboration.
- **Un seul concept par classe** : Concevez vos classes de manière à ce qu'elles aient une seule responsabilité ou un seul concept. Cela favorise la modularité et la réutilisabilité du code.
- **Documenter les classes** : Utilisez des **docstrings** pour documenter vos classes, en fournissant des descriptions claires de leur fonctionnement, de leurs attributs et de leurs méthodes. Une documentation claire facilite l'utilisation et la maintenance du code.

3. Classe et objets

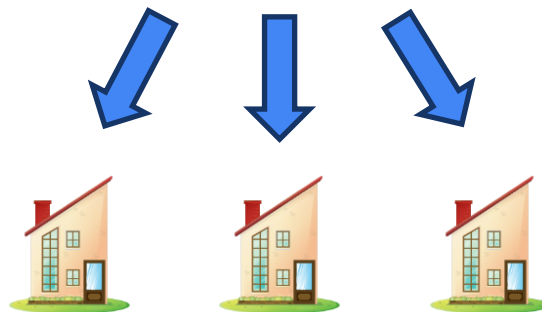
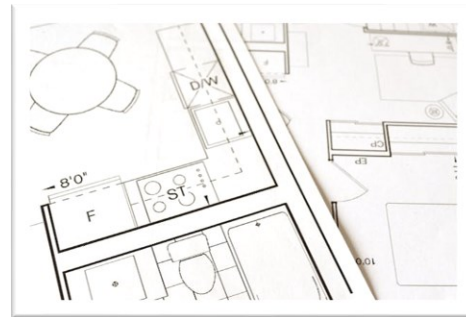
- Définition d'une classe
- Syntaxe de déclaration de classe
- Création et instanciation d'objets
- Attributs
- Méthodes

Définition d'une classe

Les classes en Python servent de modèles pour créer des objets ayant des **attributs** (états) et des **méthodes** (comportements) spécifiques.

Elles agissent comme des plans de construction ou des moules, définissant la structure et le comportement des objets qu'ellesinstancient.

Chaque objet créé à partir d'une classe est une instance de cette classe, permettant de créer autant d'objets que nécessaire avec les mêmes attributs et méthodes définis dans la classe.



Syntaxe de déclaration de classe

La syntaxe de déclaration de classe en Python est simple. Elle commence par le mot-clé `class`, suivi du nom de la classe que nous souhaiterons créer. Ce mot-clé est fondamental car il indique à Python que nous sommes sur le point de définir une nouvelle classe d'objets.

Dans cet exemple, nous avons utilisé le mot-clé `class` pour déclarer une classe nommée `Voiture`. Ceci crée une structure de classe vide, prête à être remplie avec des attributs et des méthodes spécifiques à la classe.

En utilisant la syntaxe de classe, nous pourrions définir des modèles de données et des fonctionnalités qui seront utilisés pour créer des objets de cette classe.

```
1 class Voiture:  
2     pass
```

Création et instantiation des objets

L'étape après la déclaration d'une classe est la création et l'instantiation d'objets de cette classe. Pour créer un objet à partir d'une classe en Python, nous utiliserons simplement le nom de la classe suivi de parenthèses. Cette opération est appelée instantiation.

Dans cet exemple, nous créons un nouvel objet de la classe Voiture en utilisant la syntaxe `Voiture()`. Cela crée une instance de la classe Voiture, que nous stockons dans la variable `voiture1`. L'objet `voiture1` est maintenant une instance de la classe Voiture et peut être utilisé pour accéder aux attributs et aux méthodes définis dans la classe.

```
1  # Création d'un objet de la classe Voiture
2  from Voiture import Voiture
3
4
5  voiture1 = Voiture()
```

Attributs

Les attributs représentent les caractéristiques ou les propriétés d'un objet.

Dans le contexte d'une **voiture**, les attributs pourraient inclure des informations telles que la **marque**, le **modèle**, la **couleur**, la **vitesse actuelle**, et le **nombre de portes**.

Chaque **attributs** joue un rôle essentiel dans la définition de l'état de l'objet et peut être consulté ou modifié par les méthodes de la classe associée ou par ces **attributs** en l'absence de **propriétés** qui les contrôlent.

```
1 class Voiture:
2     marque = None
3     modele = "Corolla"
4     couleur = None
5     vitesse_actuelle = 0
6
7     # Création d'une instance de voiture
8     ma_voiture = Voiture()
9
10    # Affichage des attributs de la voiture
11    print("Marque :", ma_voiture.marque)
12    print("Modèle :", ma_voiture.modele)
13    print("Couleur :", ma_voiture.couleur)
14    print("Vitesse actuelle :", ma_voiture.vitesse_actuelle)
```

Méthodes

les **méthodes** jouent un rôle essentiel en permettant aux objets d'effectuer des actions ou d'interagir avec d'autres objets. Elles représentent les comportements ou les fonctionnalités associées à un objet particulier.

Celles-ci pourraient inclure des actions telles que l'accélération, le freinage ou le calcul de la consommation de carburant.

Elles encapsulent le comportement de l'objet, offrant ainsi une interface cohérente pour manipuler et interagir avec lui.

```
1 class Voiture:
2     marque = None
3     modele = "Corolla"
4     couleur = None
5     vitesse_actuelle = 0
6
7     def accelerer(self, acceleration):
8         self.vitesse_actuelle += acceleration
9
10    def freiner(self, deceleration):
11        self.vitesse_actuelle -= deceleration
12
13    def afficher_info(self):
14        print("Marque :", self.marque if self.marque is not None else "Non définis !")
15        print("Modèle :", self.modele)
16        print("Couleur :", self.couleur if self.couleur is not None else "Non définis !")
17        print("Vitesse actuelle :", self.vitesse_actuelle)
18
19    # Création d'une instance de voiture
20    ma_voiture = Voiture()
21
22    # Affichage des informations initiales de la voiture
23    print("Informations initiales de la voiture :")
24    ma_voiture.afficher_info()
25
26    # Accélération de la voiture
27    ma_voiture.accelerer(20)
28    print("\nNouvelle vitesse après accélération :")
29    ma_voiture.afficher_info()
```

Exercices

1. Créer une class Elephant contenant :

Caractéristiques :

- nom
- appétit (sur 100)
- satisfaction (sur 100)
- en_vie
- soigneur

Comportement :

- manger()

Exercices

2. Créer une class soigneur contenant :

Caractéristiques :

- nom
- date_naissance
- expérience
- nombre_animaux_responsable

Comportement (si le soigneur est responsable de l'animal) :

- nourrir(animal)
- entretenir(animal)

Exercices

3. Créer une class enclos contenant :

Caractéristiques :

- nom
- capacite_max
- taille
- liste_animaux

Comportement :

- ajouter_animal(animal)
- enleve_animal(animal)
- afficher_animaux()
- Bonus : Imaginer un comportement permettant de simuler un passage de jour.

4. Encapsulation

- Objectif
- Déclaration avec @property
- Utilisations des méthodes getter, setter et deleter
- Propriétés en lecture seule
- Propriétés calculées
- Validation des valeurs

Objectif

Les **propriétés** offrent un moyen de contrôler l'accès aux attributs d'une classe. Elles nous permettent de définir des méthodes spéciales, telles que les **getters**, les **setters** et les **deleters**, qui seront automatiquement appelées lors de l'accès, de la modification ou de la suppression de ces attributs.

Grâce à l'utilisation de **propriétés**, il est possible de garantir l'encapsulation des données, ce qui signifie que les détails internes de la classe restent cachés à l'extérieur et ne peuvent être modifiés que de manière contrôlée via des méthodes spécifiques



Déclaration avec @property

La décoration `@property` nous permet de créer des attributs qui peuvent être accédés comme des attributs de classe, sans nécessiter l'utilisation de parenthèses.

Cette fonctionnalité permet de définir des propriétés directement dans la classe, facilitant ainsi l'accès et la gestion des données de manière transparente.

Par exemple, dans notre classe `Voiture`, nous pouvons définir une méthode `infos` décorée avec `@property`, qui permet d'accéder aux informations sur la voiture, telles que la marque, le modèle, la couleur et la vitesse actuelle, comme s'il s'agissait d'attributs de classe.

```
1 class Voiture:
2     marque = "Toyota"
3     modele = "Corolla"
4     couleur = "Rouge"
5     vitesse_actuelle = 0
6
7     @property
8     def infos(self):
9         return f"Marque: {self.marque}, ect..."
10
11 # Création d'une instance de la classe Voiture
12 ma_voiture = Voiture()
13
14 # Accès aux informations de la voiture
15 print("Informations de la voiture:")
16 print(ma_voiture.infos)
```

Utilisation des méthodes getter, setter et deleter

L'utilisation des méthodes **getter**, **setter** et **deleter** est une pratique essentielle en programmation orientée objet pour contrôler l'accès aux attributs d'une classe. Chaque méthode possède un rôle spécifique :

- **Getter (Accesseur)** : Cette méthode permet d'accéder à la valeur d'un attribut sans permettre de la modifier directement. Elle garantit un accès sécurisé et contrôlé aux données de la classe.
- **Setter (Mutateur)** : Le setter est utilisé pour modifier la valeur d'un attribut en appliquant des règles de validation ou de transformation. Il offre un moyen de modifier les données tout en maintenant l'intégrité de la classe.
- **Deleter (Suppresseur)** : Le deleter est responsable de la suppression d'un attribut de la classe. Il permet de libérer les ressources associées à l'attribut et de maintenir la cohérence de l'objet.

Utilisation des méthodes : getter

Un **getter** est une méthode utilisée pour accéder à la valeur d'un attribut d'une classe.

Celui-ci est implémenté à l'aide du décorateur **@property**, ce qui permet d'accéder à l'attribut comme s'il s'agissait d'un attribut de classe directement, sans avoir à utiliser de parenthèses pour appeler la méthode.

Ainsi, lorsque nous accédons à l'attribut `marque` de la classe `Voiture` à travers `ma_voiture.marque`, Python invoque automatiquement la méthode **getter** associée à cet attribut, permettant de récupérer sa valeur.

```
1 class Voiture:
2     _marque = "Toyota"
3
4     # Utilisation de @property pour définir un getter pour l'attribut 'marque'
5     @property
6     def marque(self):
7         """
8         Méthode getter pour l'attribut '_marque'.
9         Elle permet d'accéder à la valeur de l'attribut '_marque'.
10        """
11        return self._marque
12
13    # Utilisation de @property.getter pour définir un getter pour l'attribut 'marque'
14    @property.getter
15    def marque(self):
16        """
17        Méthode getter pour l'attribut '_marque'.
18        Elle permet également d'accéder à la valeur de l'attribut '_marque'.
19        """
20        return self._marque
21
22
23    # Création d'une instance de la classe Voiture
24    ma_voiture = Voiture()
25
26    # Accès à la marque (lecture seule)
27    print("Marque de la voiture :", ma_voiture.marque)
```

Utilisation des méthodes : setter

Un **setter** est une méthode utilisée pour modifier la valeur d'un attribut d'une classe.

Celui-ci est implémenté à l'aide du décorateur **@property.setter**. Lorsque nous attribuons une nouvelle valeur à un attribut à travers le setter, comme dans `ma_voiture.marque = "Nissan"`, Python invoque automatiquement la méthode setter associée à cet attribut, permettant de mettre à jour sa valeur.

Le setter permet de contrôler l'affectation des valeurs aux attributs, ce qui permet de valider ou de transformer les données avant de les assigner.

```
1 class Voiture:
2     _marque = "Toyota"
3
4     @property
5     def marque(self):
6         return self._marque
7
8     @marque.setter
9     def marque(self, nouvelle_marque):
10         """
11         Méthode setter pour l'attribut '_marque'.
12         Elle permet de modifier la valeur de l'attribut '_marque'.
13         """
14         print("Modification de la marque en", nouvelle_marque)
15         self._marque = nouvelle_marque
16
17
18 # Création d'une instance de la classe Voiture
19 ma_voiture = Voiture()
20
21 # Affichage de la marque initiale
22 print("Marque initiale de la voiture :", ma_voiture.marque)
23
24 # Modification de la marque à l'aide du setter
25 ma_voiture.marque = "Nissan"
26
27 # Affichage de la nouvelle marque après modification
28 print("Nouvelle marque de la voiture :", ma_voiture.marque)
```

Utilisation des méthodes : delete

Le **delete** est une fonctionnalité essentielle pour assurer la gestion complète des attributs d'une classe.

Il permet de définir un comportement spécifique lors de la suppression d'un attribut, offrant ainsi un contrôle supplémentaire sur la manipulation des données.

L'utilité du **delete** réside dans sa capacité à garantir la cohérence et l'intégrité des données de la classe, en permettant par exemple de libérer des ressources associées à un attribut lors de sa suppression.

```
1 class Voiture:
2     _marque = "Toyota"
3
4     @property
5     def marque(self):
6         return self._marque
7
8     @marque.deleter
9     def marque(self):
10        """
11        Méthode delete pour l'attribut '_marque'.
12        Elle permet de supprimer la valeur de l'attribut '_marque'.
13        """
14        print("Suppression de la marque")
15        del self._marque
16
17
18 # Création d'une instance de la classe Voiture
19 ma_voiture = Voiture()
20
21 # Affichage de la marque initiale
22 print("Marque initiale de la voiture :", ma_voiture.marque)
23
24 # Suppression de la marque à l'aide du delete
25 del ma_voiture.marque
26
27 # Affichage de la marque après suppression (devrait générer une erreur)
28 print("Marque de la voiture après suppression :", ma_voiture.marque)
```

Propriétés en lecture seule

Les propriétés en lecture seule sont des attributs dont la valeur ne peut être modifiée une fois initialisée. Elles sont définies en utilisant uniquement une méthode getter avec le décorateur `@property`, ce qui empêche toute modification directe de la valeur de la propriété.

Par exemple, dans une classe `Voiture`, nous pouvons définir une propriété en lecture seule pour l'identifiant de la voiture, garantissant ainsi que cet attribut ne peut être modifié après sa définition initiale ou via une méthode interne à la classe.

```
1 class Voiture:
2     _identifiant = "ABC123" # Attribut défini dans la classe
3
4     @property
5     def identifiant(self):
6         return self._identifiant
7
8 # Création d'une instance de la classe Voiture
9 ma_voiture = Voiture()
10
11 # Accès à la propriété en lecture seule
12 print("Identifiant de la voiture :", ma_voiture.identifiant)
13
14 # Tentative de modification de l'identifiant (ne fonctionnera pas)
15 ma_voiture.identifiant = "XYZ789" # générera une erreur
```

Propriétés calculées

Les propriétés calculées offrent la possibilité de définir des attributs dont la valeur est déterminée dynamiquement à partir d'autres attributs de l'objet.

Contrairement aux propriétés simples dont la valeur est stockée directement, les propriétés calculées sont évaluées au moment de l'accès à la propriété.

Cela permet de définir des comportements personnalisés pour l'accès aux données, en permettant de réaliser des calculs ou des transformations sur les données existantes.

```
1 class Voiture:
2     _vitesse = 0
3     _temps_ecoule = 0
4
5     @property
6     def vitesse(self):
7         return self._vitesse
8
9     @vitesse.setter
10    def vitesse(self, nouvelle_vitesse):
11        self._vitesse = nouvelle_vitesse
12
13    @property
14    def temps_ecoule(self):
15        return self._temps_ecoule
16
17    @temps_ecoule.setter
18    def temps_ecoule(self, nouveau_temps):
19        self._temps_ecoule = nouveau_temps
20
21    @property
22    def distance_parcourue(self):
23        # Calcul de la distance parcourue en fonction de la vitesse et du temps écoulé
24        return self.vitesse * self.temps_ecoule
25
26    # Création d'une instance de la classe Voiture
27    ma_voiture = Voiture()
28
29    # Attribution d'une vitesse et d'un temps écoulé
30    ma_voiture.vitesse = 80 # km/h
31    ma_voiture.temps_ecoule = 2 # heures
32
33    # Accès à la propriété calculée "distance_parcourue"
34    print("Distance parcourue par la voiture :", ma_voiture.distance_parcourue, "kilomètres")
```


Validation des valeurs

La validation des valeurs des propriétés est importante pour garantir l'intégrité des données. Les méthodes setter permettent de contrôler les valeurs avant de les attribuer à une propriété, assurant ainsi la conformité aux règles métier.

Dans notre exemple avec la classe `Voiture`, nous avons défini un validateur pour la propriété `vitesse`. Ce validateur garantit que la vitesse est positive. En cas de valeur invalide, une exception sera levée.

```
1 class Voiture:
2     def __init__(self):
3         self._vitesse = 0
4
5     @property
6     def vitesse(self):
7         return self._vitesse
8
9     @vitesse.setter
10    def vitesse(self, nouvelle_vitesse):
11        if nouvelle_vitesse < 0:
12            raise ValueError("La vitesse ne peut pas être négative.")
13        self._vitesse = nouvelle_vitesse
14
15    ma_voiture = Voiture()
16
17    # Modification de la vitesse
18    try:
19        ma_voiture.vitesse = 120 # km/h
20        print("Vitesse de la voiture :", ma_voiture.vitesse)
21    except ValueError as e:
22        print("Erreur :", e)
```

Exercices

1. Mettre en place des propriétés sur les différents attributs des classes.

Elephant :

- nom (lecture et écriture)
- rassasier (sur 100) (lecture seule)
- bonheur (sur 100) (lecture seule)
- en_vie (lecture seule)
- soigneur (lecture et écriture)

Soigneur :

- nom (lecture et écriture)
- date_naissance (lecture seule)
- expérience (lecture seule)
- nombre_animaux_responsable (lecture seule)

Enclos :

- nom (lecture et écriture)
- capacite_max (lecture et écriture)
- taille (lecture seule)
- liste_animaux (lecture seule)

Exercices

2. Mettre en place une propriété calculée qui va donner l'âge du soigneur sur base de sa date de naissance.
3. Réalisez des tests pour vérifier si votre programme continue à tourner correctement, le cas échéant appliquer les correctifs.

5. Data model

- Attributs spéciaux
- Méthodes spéciales
- opérateurs

Attributs spéciaux

Les attributs spéciaux (entourés de double underscore) sont accessibles depuis le type d'une classe ou l'instance d'une classe. Ceux-ci sont en lecture seule.

Nom	Accessible sur	Objectif
<code>__name__</code>	classe	Renvoie le nom de la classe.
<code>__bases__</code>	classe	Renvoie un tuple des classes parentes de la classe.
<code>__class__</code>	objet	Permet de connaître à partir de quelle classe l'objet a été créé.
<code>__dict__</code>	objet	Renvoie un dictionnaire des méthodes et attributs disponibles.
<code>__doc__</code>	classe et objet	Renvoie la “docstring” de la classe interrogée.

Attributs spéciaux : démonstration

```
1 class MaClasse:
2     """Ceci est la documentation de MaClasse."""
3
4     def methode(self):
5         pass
6
7     # Accéder à l'attribut __name__
8     print("Nom de la classe:", MaClasse.__name__)
9
10    # Accéder à l'attribut __bases__
11    print("Classes parentes:", MaClasse.__bases__)
12
13    # Créer un objet de la classe MaClasse
14    objet = MaClasse()
15
16    # Accéder à l'attribut __class__
17    print("Classe de l'objet:", objet.__class__)
18
19    # Accéder à l'attribut __dict__
20    print("Méthodes et attributs disponibles:", objet.__dict__)
21
22    # Accéder à l'attribut __doc__
23    print("Documentation de la classe:", MaClasse.__doc__)
```

Méthodes spéciales

Les méthodes spéciales sont des fonctions prédéfinies avec des noms spéciaux entourés de double underscore également permettent de définir le comportement d'une classe. Celle-ci peuvent être redéfinies par la classe.

Nom	Rôle
<code>__init__</code>	Permet l'initialisation des champs corrects lors de la création de l'objet.
<code>__repr__</code>	Envoie la représentation "officielle" en chaîne de caractères d'un objet.
<code>__str__</code>	Envoie la représentation "informelle" en chaîne de caractères d'un objet.
<code>__len__</code>	Renvoie la longueur de l'objet.
<code>__getitem__</code>	Permet d'accéder à un élément de l'objet en utilisant la notation d'indexation (par exemple, objet[key]).

Méthodes spéciales : démonstration

```
1 class Voiture:
2     _marque = "Honda"
3     _model = "Civic"
4     _annee = 2006
5
6     def __repr__(self):
7         return f"voiture({self._marque}, {self._model}, {self._annee})"
8
9     def __str__(self):
10        return f"{self._annee} {self._marque} {self._model}"
11
12    def __len__(self):
13        # Définir la longueur de la voiture
14        return len(str(self))
15
16    def __getitem__(self, key):
17        # Permet d'accéder à un attribut spécifique de la voiture
18        if key == 'marque':
19            return self._marque
20        elif key == 'model':
21            return self._model
22        elif key == 'annee':
23            return self._annee
24        else:
25            raise KeyError(f"Attribut '{key}' non valide pour la voiture")
```

```
27 # Création d'une voiture
28 voiture = Voiture()
29
30 # Affichage de la représentation officielle et informelle de la voiture
31 print(repr(voiture)) # Affiche la représentation officielle
32 print(str(voiture))  # Affiche la représentation informelle
33
34 # Obtention de la longueur de la voiture
35 print(len(voiture))
36
37 # Accès aux attributs de la voiture en utilisant la notation d'indexation
38 print(voiture['marque'])
39 print(voiture['model'])
40 print(voiture['annee'])
```


Opérateurs spéciaux

Il sera possible également de manipuler les opérateurs afin de redéfinir le comportement lors de la comparaison.

Méthode spéciale	Rôle	code classique
<code>__eq__</code>	Renvoie True si les objets sont égaux, False sinon.	<code>obj1 == obj2</code>
<code>__lt__</code>	Renvoie True si l'objet actuel est strictement inférieur à l'autre objet, False sinon.	<code>obj1 < obj2</code>
<code>__le__</code>	Renvoie True si l'objet actuel est inférieur ou égal à l'autre objet, False sinon.	<code>obj1 <= obj2</code>
<code>__gt__</code>	Renvoie True si l'objet actuel est strictement supérieur à l'autre objet, False sinon.	<code>obj1 > obj2</code>
<code>__ge__</code>	Renvoie True si l'objet actuel est supérieur ou égal à l'autre objet, False sinon.	<code>obj1 >= obj2</code>

Opérateurs spéciaux : démonstration

```
1 class Car:
2     _marque = ""
3     _model = ""
4     _annee = 0
5     ...
6     def __eq__(self, other): # Opérateur ==
7         return self._marque == other._marque and self._model == other._model and self._annee == other._annee
8     ...
9     def __ne__(self, other): # Opérateur !=
10        return not self == other
11    ...
12    def __lt__(self, other): # Opérateur <
13        if self._annee != other._annee:
14            return self._annee < other._annee
15        elif self._marque != other._marque:
16            return self._marque < other._marque
17        else:
18            return self._model < other._model
19    ...
20    def __le__(self, other): # Opérateur <=
21        return self < other or self == other
22    ...
23    def __gt__(self, other): # Opérateur >
24        return not (self < other or self == other)
25    ...
26    def __ge__(self, other): # Opérateur >=
27        return not (self < other)
```

```
29 # Création de quelques voitures
30 car1 = Car()
31 car1._marque = "Toyota"
32 car1._model = "Camry"
33 car1._annee = 2020
34
35 car2 = Car()
36 car2._marque = "Honda"
37 car2._model = "Accord"
38 car2._annee = 2019
39
40 car3 = Car()
41 car3._marque = "Toyota"
42 car3._model = "Corolla"
43 car3._annee = 2020
44
45 # Comparaison des voitures
46 print(car1 == car2) # False
47 print(car1 != car2) # True
48 print(car1 < car2) # True
49 print(car1 > car2) # False
50 print(car1 <= car3) # True
51 print(car2 >= car3) # False
```

6. Heritage

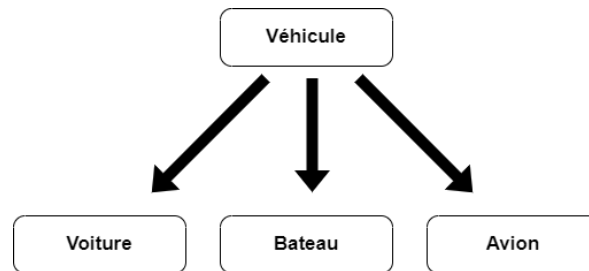
- Définition
- Héritage simple
- La méthode << Super() >>
- Héritage multiple
- Redéfinition de méthodes
- Trucs et astuces

Définition de l'héritage

L'héritage est un concept fondamental en programmation orientée objet, offrant la possibilité aux classes de partager des attributs et des méthodes avec d'autres classes, appelées superclasses.

Cette relation parent-enfant permet aux sous-classes d'hériter et d'étendre les fonctionnalités de la superclasse, favorisant ainsi la réutilisabilité du code et la modularité de la conception logicielle.

En héritant des caractéristiques de la superclasse, les sous-classes peuvent bénéficier d'une implémentation prédéfinie, tout en ayant la flexibilité d'ajouter leurs propres fonctionnalités spécifiques.



Heritage simple

Pour mettre en place l'héritage, nous devons suivre une structure claire et méthodique. Tout d'abord, nous définissons la classe parente, qui contient les attributs et les méthodes communs à plusieurs classes.

Ensuite, nous déclarons la classe enfant en spécifiant entre parenthèses le nom de la classe parente, indiquant ainsi l'héritage. Dans la sous-classe, nous pouvons personnaliser le comportement en ajoutant de nouveaux attributs ou en définissant des nouvelles méthodes.

```
1 class Vehicule:
2     marque = "honda"
3     model = "Civic"
4     def afficher_info(self):
5         print("Marque :", self.marque)
6         print("Modèle :", self.model)
7
8
9 class Voiture(Vehicule):
10     couleur = None
11
12
13 # Création d'une instance de la classe Voiture
14 ma_voiture = Voiture()
15 ma_voiture.marque = "Toyota"
16 ma_voiture.model = "Corolla"
17 ma_voiture.couleur = "rouge"
18
19 # Affichage des informations de la voiture
20 print("Informations de la voiture :")
21 ma_voiture.afficher_info()
```

La méthode << Super() >>

La méthode `super()` permet d'accéder aux méthodes et aux attributs de la classe parente à partir de la classe enfant.

Cela est particulièrement utile lorsque nous redéfinissons des méthodes dans la sous-classe tout en souhaitant appeler la méthode équivalente de la classe parente.

`super()` permet donc d'appeler la méthode parente de manière dynamique et flexible, sans avoir à spécifier explicitement le nom de la classe parente.

```
9  class Voiture(Vehicule):
10      couleur = "Rouge"
11
12      def afficher_info(self):
13          super().afficher_info()
14          print("Couleur :", self._couleur)
```

Heritage multiple

En Python, contrairement à d'autres langages, il est possible d'appliquer l'héritage multiple. Ce qui signifie qu'une classe peut hériter des attributs et des méthodes de plusieurs autres classes.

Cela permet à une classe enfant d'avoir accès aux fonctionnalités de plusieurs classes parentes.

```
1 class Vehicule:
2     marque = None
3
4     def afficher_marque(self):
5         print("Marque du véhicule :", self.marque)
6
7
8 class Modele:
9     model = None
10
11     def afficher_modele(self):
12         print("Modèle du véhicule :", self.model)
13
14
15 class Voiture(Vehicule, Modele):
16
17     def afficher_info(self):
18         self.afficher_marque()
19         self.afficher_modele()
20
21
22 # Création d'une instance de la classe Voiture
23 ma_voiture = Voiture()
24 ma_voiture.marque = "Toyota"
25 ma_voiture.model = "Corolla"
26
27 # Affichage des informations de la voiture
28 ma_voiture.afficher_info()
```

Redéfinition de méthodes

La redéfinition de méthode permet de redéfinir une méthode existante dans la classe parente dans une classe enfant, en fournissant une implémentation spécifique à cette classe enfant.

Cela permet à la classe enfant de modifier ou d'étendre le comportement de la méthode héritée de la classe parente sans modifier la classe parente elle-même.

Cette technique est utile pour personnaliser le comportement des méthodes en fonction des besoins spécifiques de chaque classe enfant.

```
1 class Vehicule:
2     def demarrer(self):
3         print("Le véhicule démarre.")
4
5     def arreter(self):
6         print("Le véhicule s'arrête.")
7
8
9 class Voiture(Vehicule):
10     def demarrer(self):
11         print("La voiture démarre en appuyant sur l'accélérateur.")
12
13     # La méthode arreter() n'est pas redéfinie, donc elle est héritée de la classe parente
14
15
16 # Création d'une instance de la classe Voiture
17 ma_voiture = Voiture()
18
19 # Appel des méthodes surchargées
20 ma_voiture.demarrer() # Affiche: La voiture démarre en appuyant sur l'accélérateur.
21 ma_voiture.arreter()  # Affiche: Le véhicule s'arrête.
```


Trucs et astuces : `issubclass()`

La fonction `issubclass()` permet de vérifier si une classe est une sous-classe d'une autre classe. Elle prend deux arguments : la classe enfant et la classe parente, et renvoie `True` si la première est une sous-classe de la seconde, sinon `False`.

Cette fonction est utile pour vérifier les relations d'héritage entre les classes et peut être utilisée dans divers contextes, comme la gestion des exceptions ou la définition de comportements spécifiques basés sur la hiérarchie des classes.

```
1  class Vehicule:
2      pass
3
4  class Voiture(Vehicule):
5      pass
6
7  class Moto(Vehicule):
8      pass
9
10 # Vérification si Voiture est une sous-classe de Vehicule
11 print(issubclass(Voiture, Vehicule)) # Output: True
12
13 # Vérification si Moto est une sous-classe de Vehicule
14 print(issubclass(Moto, Vehicule))    # Output: True
15
16 # Vérification si Vehicule est une sous-classe de Moto
17 print(issubclass(Vehicule, Moto))    # Output: False
```

Exercices

1. Créez de la classe Girafe qui reprendra les mêmes éléments que la classe Éléphant mais qui implémentera maintenant en plus les éléments suivants :

Caractéristiques :

- longueur_cou (lecture & écriture)

Comportement :

- manger_feuilles()
- boire_eau()

Exercices

2. Adaptez la classe Éléphant afin qu'elle contienne à présent les éléments suivants :

Caractéristiques :

- longueur_defense (lecture & ecriture)

Comportement :

- prendre_bain_de_boue()
- aspirer_eau()

Exercices

3. Mettez en place le concept d'héritage en créant la classe `Animal` qui reprendra les éléments communs aux classes `Girafe` et `Éléphant`.
4. Définissez une méthode `observer_environnements()` au sein de cette classe `Animal` et redéfinissez-la ensuite dans les enfants pour adapter son implémentation.
5. Réalisez des tests en implémentant les différentes fonctionnalités ajoutées et vérifiez leurs bon fonctionnement.

7. Polymorphisme

Le polymorphisme en Python permet à des objets de différentes classes d'être traités de manière uniforme s'ils partagent une interface commune.

Cela signifie que des méthodes portant le même nom peuvent agir différemment selon le type de l'objet sur lequel elles sont appelées.

```
1 class Voiture:
2     def description(self):
3         return "Je suis une voiture standard."
4
5 class VoitureSport(Voiture):
6     def description(self):
7         return "Je suis une voiture de sport rapide."
8
9 class VoitureElectrique(Voiture):
10    def description(self):
11        return "Je suis une voiture électrique économe en carburant."
12
13 # Fonction générique prenant une voiture en argument
14 def afficher_description(voiture):
15     print(voiture.description())
16
17 # Création d'instances de différentes voitures
18 voiture_standard = Voiture()
19 voiture_sport = VoitureSport()
20 voiture_electrique = VoitureElectrique()
21
22 # Appel de la fonction avec différentes instances de voitures
23 afficher_description(voiture_standard)
24 afficher_description(voiture_sport)
25 afficher_description(voiture_electrique)
```

Exercices

1. Mettre en place une méthode `comportement_hasard()` dans les classes girafe et éléphant qui permettront de choisir au hasard un des comportement propre à chaque animal.
2. Réalisez des tests afin d'appeler la méthode l'instance sur différents objets.

8. Classes abstraites

- Objectif
- Module abc
- Décorateurs abstraits
- Implémentations

Objectif

Les classes abstraites servent de modèles pour d'autres classes. Elles peuvent contenir des méthodes et des propriétés abstraites, qui sont des éléments déclarés mais non implémentés.

Ces classes ne peuvent pas être instanciées directement, mais sont conçues pour être sous-classées afin d'implémenter les méthodes et les propriétés abstraites nécessaires.

Les méthodes abstraites définissent des comportements que les sous-classes doivent fournir, tandis que les propriétés abstraites spécifient les attributs que les sous-classes doivent posséder.



module abc

Le module `abc`, abréviation de "Abstract Base Classes", est un module intégré à Python qui fournit des fonctionnalités pour la définition de classes abstraites et l'implémentation de l'héritage d'interface.

```
1  from abc import ABC
2
3  class Vehicule(ABC):
4      def mouvement(self):
5          pass
```

décorateurs abstraits

Les décorateurs abstraits du module `abc` stipulé dans le slide précédent sont utilisés pour définir des classes abstraites ainsi que des méthodes et des propriétés abstraites de manière explicite.

Ces décorateurs permettent de marquer une classe comme étant abstraite et de déclarer des méthodes et des propriétés comme abstraites, sans fournir d'implémentation.

Cela permet de créer une structure cohérente pour des classes destinées à être sous-classées, en spécifiant les comportements attendus sans les implémenter directement.

```
1  from abc import ABC, abstractmethod, abstractproperty
2
3  class Vehicule(ABC):
4      @abstractproperty
5      def vitesse_max(self):
6          pass
7
8      @abstractmethod
9      def mouvement(self):
10         pass
```

Implémentations

Lorsqu'une classe abstraite définit des méthodes abstraites et des propriétés abstraites, toute sous-classe doit fournir une implémentation de ces membres pour être considérée comme complète.

En implémentant ces méthodes et propriétés abstraites, les sous-classes donnent vie aux fonctionnalités et caractéristiques définies dans la classe abstraite, en les adaptant à leur propre contexte et en fournissant une logique spécifique à leur domaine d'application.

```
1  from abc import ABC, abstractmethod, abstractproperty
2
3  class Vehicule(ABC):
4      @abstractproperty
5      def vitesse_max(self):
6          pass
7
8      @abstractmethod
9      def mouvement(self):
10         pass
11
12  class Voiture(Vehicule):
13      _vitesse_actuelle = 0
14
15      @property
16      def vitesse_actuelle(self):
17          return self._vitesse_actuelle
18
19      @property
20      def vitesse_max(self):
21          return 200
22      ...
23      def mouvement(self):
24          return "La voiture roule"
25
26  ma_voiture = Voiture()
27
28  print("Vitesse actuelle :", ma_voiture.vitesse_actuelle)
29  print("Vitesse maximale :", ma_voiture.vitesse_max)
30
31  print(ma_voiture.mouvement())
```

Exercices

1. Modifier la classe `Animal` en classe abstraite.
2. Changer l'implémentation des méthodes `observer_environnement()` et `ramasser_objet()` en méthodes abstraites, avec toutes les implications que cela comporte.
3. Implémenter une nouvelle méthode abstraite `probabilite_deces()` dans la classe `Animal` et définir ce qui manque dans le programme pour que celui-ci continue à fonctionner.
4. Réaliser des tests pour vérifier le bon fonctionnement des nouveaux comportements.

9. Membres statiques

- Objectif
- Définition

Objectif

Les classes statiques sont des entités au sein d'une classe qui sont partagées par toutes les instances de cette classe, plutôt que par chaque instance individuellement.

Elles peuvent contenir des attributs ou des méthodes qui ne dépendent pas de l'état spécifique de chaque instance, mais sont plutôt liés à la classe dans son ensemble.

Ces membres peuvent être accessibles sans avoir besoin d'instancier la classe, ce qui les rend utiles pour les fonctions utilitaires, les constantes partagées, ou pour encapsuler des fonctionnalités qui ne dépendent pas de l'état de l'objet.

⚠ Contrairement à d'autres langages (telle que le C# et le Java), le python permet d'accéder aux membres statiques depuis une instance de la classe !

Définition

Par exemple, la classe `Voiture` définit deux méthodes statiques, `verifier_immatriculation` et `verifier_annee_fabrication`, qui sont utilisées pour valider respectivement une immatriculation et une année de fabrication de voiture.

Ces méthodes peuvent être appelées directement à partir de la classe sans nécessiter d'instanciation d'objet. Contrairement aux méthodes d'instance, celles-ci ne possèdent pas l'argument « *self* ».

```
1 class Voiture:
2     @staticmethod
3     def verifier_immatriculation(immatriculation):
4         return immatriculation.isalnum() and len(immatriculation) == 7
5
6     @staticmethod
7     def verifier_annee_fabrication(annee):
8         return 1900 <= annee <= 2024
9
10    immatriculation_valide = Voiture.verifier_immatriculation("AB123CD")
11    annee_valide = Voiture.verifier_annee_fabrication(2010)
12
13    print("Immatriculation valide :", immatriculation_valide)
14    print("Année de fabrication valide :", annee_valide)
```

Définition

Bien que nous n'ayons pas encore explicitement étudié les constructeurs, nous avons déjà appliqué le concept d'attributs statiques dans les chapitres précédents.

L'attribut `nombre_total_voitures` de la classe `Voiture` agit comme un compteur global.

Cet attribut est automatiquement mis à jour pour refléter le nombre total de voitures créées jusqu'à présent.

Les constructeurs seront abordés dans un chapitre ultérieur.

```
1 class Voiture:
2     nombre_total_voitures = 0
3
4     def __init__(self, marque, modele):
5         self.marque = marque
6         self.modele = modele
7         Voiture.nombre_total_voitures += 1
8
9     @staticmethod
10    def afficher_nombre_total_voitures():
11        print("Nombre total de voitures :", Voiture.nombre_total_voitures)
12
13    voiture1 = Voiture("Toyota", "Camry")
14    voiture2 = Voiture("Honda", "Civic")
15    voiture3 = Voiture("Ford", "Fusion")
16
17    Voiture.afficher_nombre_total_voitures()
```


Exercices

1. Définir une classe statique Outils qui comportera les méthodes suivantes :

- Clear_console() => nettoie la console
- Pause(seconde) => simule un temps de pause

2. Adaptez l'appel dans le programme.

10. Interfaces

- Objectif
- Duck Typing
- Définition
- Trucs et astuces

Objectif

Les interfaces en programmation orientée objet servent à définir un contrat entre les classes, spécifiant les méthodes qu'elles doivent implémenter.

L'objectif principal des interfaces est de garantir une certaine cohérence dans l'utilisation des classes en fournissant une structure commune pour les fonctionnalités partagées.

Elles permettent également de favoriser la modularité du code en nous permettant de travailler avec des abstractions plutôt que des implémentations concrètes.



Duck typing

Le duck typing, concept de typage en programmation, il tire son nom du test du canard : si un objet ressemble à un certain type par ses caractéristiques et son comportement, alors il est considéré comme ce type, indépendamment de son héritage formel.

En Python, cela signifie qu'un objet est considéré comme appartenant à un type si celui-ci implémente les méthodes spécifiques à ce type, sans nécessiter de déclaration explicite d'appartenance à ce type.

Par exemple, un objet est considéré comme une collection s'il implémente les méthodes spécifiques telles que `len`, `contains`, et `iter`, même s'il n'est pas explicitement déclaré comme une instance de la classe `Collection`.



Définition

la définition d'interfaces explicites n'est pas possible en python. Cependant, nous pouvons les simuler en utilisant des mécanismes tels que les classes abstraites, l'héritage multiple et le polymorphisme.

Ces approches permettent de définir un ensemble de méthodes que les classes doivent implémenter pour être considérées comme conformes à une interface donnée.

Définition

La première étape consiste à créer une interface en définissant une classe abstraite avec les méthodes souhaitées, sans fournir leur implémentation. Cela crée un contrat que les classes devront respecter.

```
1  from abc import ABC, abstractmethod
2
3  class Vehicule(ABC):
4      @abstractmethod
5      def accelerer(self, acceleration):
6          pass
7
8      @abstractmethod
9      def freiner(self, deceleration):
10         pass
```

Définition

Ensuite, nous mettons en place l'héritage entre cette interface et la classe concrète que nous voulons adapter à l'interface. Cela lie notre classe à celle-ci, garantissant que notre classe implémente toutes les méthodes requises.

Enfin, nous implémentons les méthodes de l'interface dans notre classe concrète. Cela signifie que nous fournissons une implémentation spécifique à chaque méthode abstraite définie dans l'interface, adaptée au comportement attendu de notre classe.

```
1  from abc import ABC, abstractmethod
2
3  class Vehicule(ABC):
4      @abstractmethod
5      def accelerer(self, acceleration):
6          pass
7
8      @abstractmethod
9      def freiner(self, deceleration):
10         pass
11
12  class Voiture(Vehicule):
13      def accelerer(self, acceleration):
14          print(f"La voiture accélère de {acceleration} km/h")
15
16      def freiner(self, deceleration):
17          print(f"La voiture freine de {deceleration} km/h")
```

Trucs et astuces

Le polymorphisme permet d'utiliser la méthode `isinstance()` pour vérifier si un objet est une instance d'une interface spécifique. En passant l'objet à tester et l'interface comme arguments, cette méthode retourne `True` si l'objet implémente l'interface, sinon `False`.

```
1  from abc import ABC, abstractmethod
2
3  class Interface(ABC):
4      @abstractmethod
5      def methode_interface(self):
6          pass
7
8  class MaClasse(Interface):
9      def methode_interface(self):
10         print("Méthode de l'interface implémentée")
11
12  objet = MaClasse()
13
14  if isinstance(objet, Interface):
15      print("L'objet implémente l'interface.")
16  else:
17      print("L'objet n'implémente pas l'interface.")
```


Exercices

1. Mettez en place une interface IAnimal qui devra reprendre les comportements suivants :

- manger()
- observer_environnement()
- faire_une_sieste()
- probabilite_deces()
- diminuer_rassasier()
- diminuer_bonheur()

2. Implémentez l'interface au niveau des classes Éléphant et Girafe.

3. Agissez en conséquence pour que le programme continue à fonctionner.